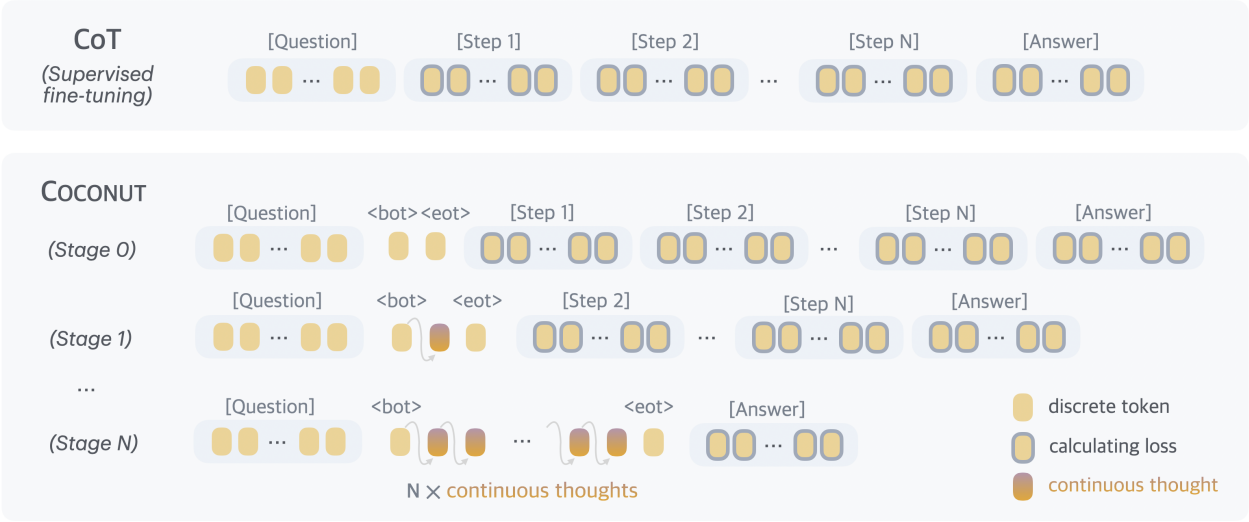

Figure 2 Training procedure of Chain of Continuous Thought (COCONUT). Given training data with language reasoning steps, at each training stage we integrate c additional continuous thoughts ($c = 1$ in this example), and remove one language reasoning step. The cross-entropy loss is then used on the remaining tokens after continuous thoughts.

generating the next token. In latent mode, it directly utilizes the last hidden state as the next input embedding. This last hidden state represents the current reasoning state, termed as a "continuous thought".

Special tokens <bot> and <eot> are employed to mark the beginning and end of the latent thought mode, respectively. As an example, we assume latent reasoning occurs between positions i and j, i.e., $x_i =$ <bot> and $x_j =$ <eot>. When the model is in the latent mode ($i < t < j$), we use the last hidden state from the previous token to replace the input embedding, i.e., $E_t = [e(x_1), e(x_2), ..., e(x_i), h_i, h_{i+1}, ..., h_{t-1}]$. After the latent mode finishes ($t \geq j$), the input reverts to using the token embedding, i.e., $E_t = [e(x_1), e(x_2), ..., e(x_i), h_i, h_{i+1}, ..., h_{j-1}, e(x_j), ..., e(x_t)]$. It is worth noting that the last hidden states have been processed by the final normalization layer, so they are not too large in magnitude. $\mathcal{M}(x_{t+1} \mid x_{\leq t})$ is not defined when $i < t < j$, since the latent thought is not intended to be mapped back to language space. However, $\text{softmax}(Wh_t)$ can still be calculated for probing purposes (see Section 5).

Training Procedure. In this work, we focus on a problem-solving setting where the model receives a question as input and is expected to generate an answer through a reasoning process. We leverage language CoT data to supervise continuous thought by implementing a multi-stage training curriculum inspired by Deng et al. (2024). As shown in Figure 2, in the initial stage, the model is trained on regular CoT instances. In the subsequent stages, at the k-th stage, the first k reasoning steps in the CoT are replaced with $k \times c$ continuous thoughts[1], where c is a hyperparameter controlling the number of latent thoughts replacing a single language reasoning step. Following Deng et al. (2024), we also reset the optimizer state when training stages switch. We insert <bot> and <eot> tokens (which are not counted towards c) to encapsulate the continuous thoughts.

During the training process, we optimize the normal negative log-likelihood loss, but mask the loss on questions and latent thoughts. It is important to note that the objective does **not** encourage the continuous thought to *compress the removed language thought*, but rather to *facilitate the prediction of future reasoning*. Therefore, it's possible for the LLM to learn more effective representations of reasoning steps compared to human language.

Training Details. Our proposed continuous thoughts are fully differentiable and allow for back-propagation. We perform $n + 1$ forward passes when n latent thoughts are scheduled in the current training stage, computing a new latent thought with each pass and finally conducting an additional forward pass to obtain a loss on the remaining text sequence. While we can save any repetitive computing by using a KV cache, the sequential nature of the multiple forward passes poses challenges for parallelism. Further optimizing the

[1]If a language reasoning chain is shorter than k steps, then all the language thoughts will be removed.

training efficiency of COCONUT remains an important direction for future research.

Inference Process. The inference process for COCONUT is analogous to standard language model decoding, except that in latent mode, we directly feed the last hidden state as the next input embedding. A challenge lies in determining when to switch between latent and language modes. As we focus on the problem-solving setting, we insert a `<bot>` token immediately following the question tokens. For `<eot>`, we consider two potential strategies: a) train a binary classifier on latent thoughts to enable the model to autonomously decide when to terminate the latent reasoning, or b) always pad the latent thoughts to a constant length. We found that both approaches work comparably well. Therefore, we use the second option in our experiment for simplicity, unless specified otherwise.

4 Continuous Space Enables Latent Tree Search

In this section, we provide a proof of concept of the advantage of continuous latent space reasoning. On ProsQA, a new dataset that requires extensive planning ability, COCONUT outperforms language space CoT reasoning. Interestingly, our analysis indicates that the continuous representation of reasoning can encode multiple alternative next reasoning steps. This allows the model to perform a breadth-first search (BFS) to solve the problem, instead of prematurely committing to a single deterministic path like language CoT.

We start by introducing the experimental setup (Section 4.1). By leveraging COCONUT's ability to switch between language and latent space reasoning, we are able to control the model to interpolate between fully latent reasoning and fully language reasoning and test their performance (Section 4.2). This also enables us to interpret the latent reasoning process as tree search (Section 4.3). Based on this perspective, we explain why latent reasoning can help LLMs make better decisions (Section 4.4).

4.1 Experimental Setup

Dataset. We introduce ProsQA (Proof with Search Question-Answering), a new logical reasoning dataset. A visualized example is shown in Figure 4. Each instance in ProsQA consists of a directed acyclic graph (DAG) of logical relationships between concepts, presented as natural language statements. The task requires models to determine logical relationships by finding valid paths through this graph, demanding sophisticated planning and search strategies. Unlike previous logical reasoning datasets like ProntoQA (Saparov and He, 2022), ProsQA's DAG structure introduces complex exploration paths, making it particularly challenging for models to identify the correct reasoning chain. More comprehensive details about the dataset construction and characteristics can be found in Appendix A.

Setup. We use a pre-trained GPT-2 model as the base model for all experiments. The learning rate is set to 1×10^{-4} while the effective batch size is 128. We train a COCONUT model following the training procedure in Section 3. Since the maximum reasoning steps in ProsQA is 6, we set the number of training stages to $N = 6$ in the training procedure. In each stage, we train the model for 5 epochs, and stay in the last stage until the 50 epochs. The checkpoint with the best accuracy in the last stage is used for evaluation. As reference, we report the performance of (1) *CoT*: the model is trained with CoT data, and during inference, the model will generate a complete reasoning chain to solve the problem. (2) *no-CoT*: the model is trained with only the question and answer pairs, without any reasoning steps. During inference, the model will output the final answer directly.

To understand the properties of latent and language reasoning space, *we manipulate the model to switch between fully latent reasoning and fully language reasoning*, by manually setting the position of the `<eot>` token during inference. When we enforce COCONUT to use k continuous thoughts, the model is expected to output the remaining reasoning chain in language, starting from the $k + 1$ step. In our experiments, we test variants of COCONUT on ProsQA with $k \in \{0, 1, 2, 3, 4, 5, 6\}$. Note that all these variants only differ in inference time while sharing the same model weights.

Metrics. We apply two sets of evaluation metrics. One of them is based on the correctness of the *final answer*, regardless of the reasoning process. It is also the main metric used in the later sections (Section 5.3). To enable fine-grained analysis on ProsQA, we define another metric on the *reasoning process*. We classify a reasoning chain into (1) **Correct Path**: The output is one of the shortest paths to the correct answer. (2)

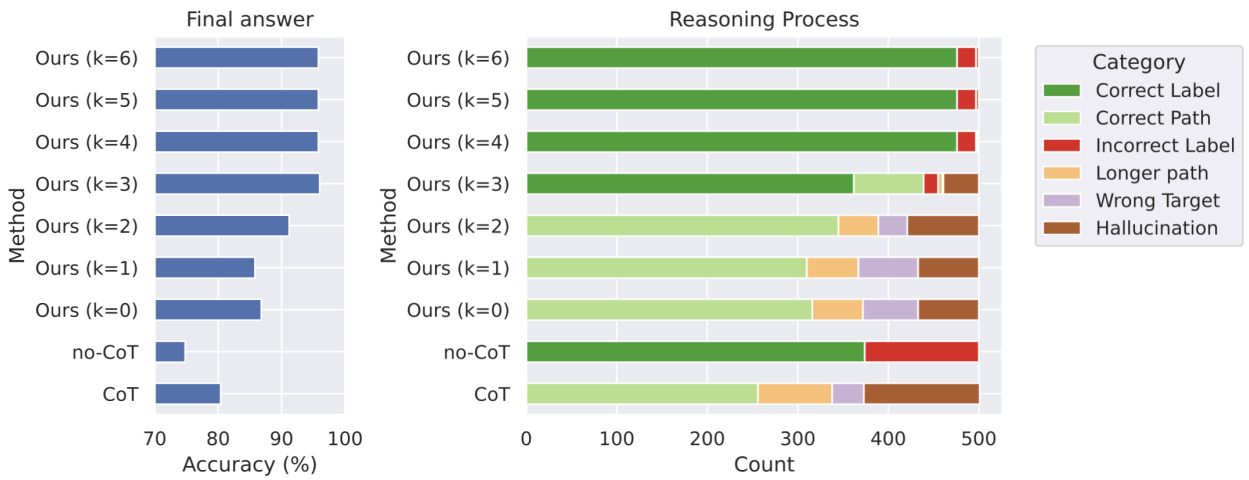

Figure 3 The accuracy of final answer (left) and reasoning process (right) of multiple variants of COCONUT and baselines on ProsQA.

Longer Path: A valid path that correctly answers the question but is longer than the shortest path. (3) **Hallucination**: The path includes nonexistent edges or is disconnected. (4) **Wrong Target**: A valid path in the graph, but the destination node is not the one being asked. These four categories naturally apply to the output from COCONUT ($k = 0$) and *CoT*, which generate the full path. For COCONUT with $k > 0$ that outputs only partial paths in language (with the initial steps in continuous reasoning), we classify the reasoning as a Correct Path *if a valid explanation can complete it*. Also, we define Longer Path and Wrong Target for partial paths similarly. If no valid explanation completes the path, it's classified as Hallucination. In *no-CoT* and COCONUT with larger k, the model may only output the final answer without any partial path, and it falls into (5) **Correct Label** or (6) **Incorrect Label**. These six categories cover all cases without overlap.

4.2 Overall Results

Figure 3 presents a comparative analysis of various reasoning methods evaluated on ProsQA. The model trained using *CoT* frequently hallucinates non-existent edges or outputs paths leading to incorrect targets, resulting in lower answer accuracy. In contrast, COCONUT, which leverages continuous space reasoning, demonstrates improved accuracy as it utilizes an increasing number of continuous thoughts. Additionally, the rate of correct reasoning processes (indicated by "Correct Label" and "Correct Path") significantly increases. At the same time, there is a notable reduction in instances of "Hallucination" and "Wrong Target," issues that typically emerge when the model makes mistakes early in the reasoning process.

An intuitive demonstration of the limitations of reasoning in language space is provided by the case study depicted in Figure 4. As shown, models operating in language space often fail to plan ahead or backtrack. Once they commit to an incorrect path, they either hallucinate unsupported edges or terminate with irrelevant conclusions. In contrast, latent reasoning avoids such premature commitments by enabling the model to iteratively refine its decisions across multiple reasoning steps. This flexibility allows the model to progressively eliminate incorrect options and converge on the correct answer, ultimately resulting in higher accuracy.

4.3 Interpreting the Latent Reasoning as Tree Search

To better understand COCONUT, we probe the latent reasoning process by forcing the model to explicitly generate language reasoning steps following intermediate continuous thoughts (Figure 5). Using the example presented in Figure 4, at the initial reasoning step, the model must select which immediate child node of "Alex" to consider next, specifically from the set {"lempus", "sterpus", "zhorpus", "grimpus"}. The distribution over these candidate next steps is visualized in Figure 5, left. In the subsequent reasoning step, these nodes expand further into an extended set of potential paths, including all grandchildren of "Alex" (Figure 5, right).